DAY 2 OF 7

Token-based login & roles

Register and log in through JSON APIs, then protect every route by token and role

Today's plan

10 min	Recap of Day 1, run the app
20 min	Concepts: authentication, tokens, roles, CORS
50 min	Live code: register, login, logout, me
15 min	JSON error handlers and CORS
20 min	Test every API with curl
5 min	Wrap-up

By the end you can

- Explain session login vs token login
- Protect routes with @auth_required and @roles_required
- Explain why the browser needs CORS in our setup

Authentication vs authorization

Authentication

- Who are you?
- Email + password checked once
- Server answers with a token
- Fails with 401 Unauthorized

Authorization

- What may you do?
- Checked on every request
- Based on the role: admin or user
- Fails with 403 Forbidden

How token login works



The server keeps no session. Logout changes fs_uniquifier, which makes every old token invalid.

What Flask-Security-Too gives us



hash / verify

hash_password() and verify_password() – bcrypt, never plain text



get_auth_token()

Creates the token returned after a correct login



@auth_required

"token" – rejects missing or bad tokens with 401



@roles_required

"admin" – rejects other roles with 403

The login API

backend/application/api/auth.py

```
@auth_bp.post("/login")
def login():
    data, missing = get_json("email", "password")
    if missing:
        return error("Email and password are required")

    user = datastore.find_user(email=data["email"].lower())
    ok = user and verify_password(data["password"],
                                  user.password)
    if not ok:
        return error("Invalid email or password", 401)
    if not user.active:
        return error("Your account is blocked.", 403)

    user.last_visit = datetime.utcnow()
    db.session.commit()
    return jsonify(token=user.get_auth_token(),
                   user=user_to_dict(user))
```



Validate input

get_json() trims values and lists missing fields



Same message

wrong email or password → one 401, no hints



Blocked users

cannot log in (used on Day 5)

Protecting a route

backend/application/api/auth.py

```
@auth_bp.get("/me")
@auth_required("token")
def me():
    return jsonify(user=user_to_dict(current_user))


@auth_bp.get("/admin/ping")
@auth_required("token")
@roles_required("admin")
def admin_ping():
    name = current_user.full_name
    return jsonify(message=f"Hello admin {name}")
```



current_user

the user the token belongs to



Order matters

token check first, then the role check



JSON errors

custom handlers return {"message": …} instead of HTML

Why CORS?



Different port = different origin. flask-cors tells the browser that only http://localhost:5173 may call /api/*, with the Authentication-Token header.

Test with curl

Terminal

```
curl -X POST localhost:5000/api/register \
  -H "Content-Type: application/json" \
  -d '{"email":"a@b.com","password":"secret1",
       "full_name":"Ann"}'

curl -X POST localhost:5000/api/login \
  -H "Content-Type: application/json" \
  -d '{"email":"a@b.com",
       "password":"secret1"}'

curl localhost:5000/api/me \
  -H "Authentication-Token: <token>"
```

Check that...

- /api/me without a token → 401
- /api/me with the token → your user
- /api/admin/ping as user → 403, as admin → 200
- After /api/logout the old token → 401

Common errors

Error	Fix
Always 401 even with a token	header name must be exactly Authentication-Token
HTML redirect instead of JSON	add the unauthn_handler / unauthz_handler in app.py
Error mentioning bcrypt	pip install bcrypt; keep SECURITY_PASSWORD_HASH = "bcrypt"
CORS error in the browser console	FRONTEND_ORIGIN must match the Vue URL exactly

Recap

- Login returns a token; the frontend sends it on every request
- 401 = not logged in, 403 = logged in but not allowed
- Logout rotates fs_uniquifier so old tokens stop working
- CORS lets only our Vue origin call the API

Next: Day 3 – REST APIs for subjects, chapters, quizzes, questions and scores.